BÁO CÁO KIỂM THỬ: FEATURE D — MOBILE CART (GIỎ HÀNG ỨNG DỤNG DI ĐỘNG)

- **Phân hệ (Pool):** Pool D — Mobile App
- **Chức năng:** Mobile Shopping Cart & Checkout Flow (FR-07 / FR-08 trên Mobile)
- **Nền tảng kiểm thử:** React Native / Expo Mobile App (`Seminar/src/eshop-sut/frontend-mobile/App.js`) & Backend API (`http://localhost:3000/api`)
- **Phương pháp thực thi: Kiểm thử hộp trắng / Phân tích mã nguồn tĩnh (White-box Code Inspection & Static Analysis)** kết hợp **Kiểm thử tích hợp Backend API (Backend API Integration Testing)**.
- **Tiêu chuẩn học thuật áp dụng:**
  - **Slide S04:** Domain Testing & Boundary Value Analysis (BVA)
  - **Slide S07:** Test Case Essentials, Objective Syntax (`Action + Function + Operating Condition`) & Template 6 cột
  - **Slide S08:** Bug Report 10 Essentials & Test Summary Report 3-table Statistics

> [!IMPORTANT] **Tuyên Bố Phương Pháp Luận & Tính Minh Bạch Môi Trường Thực Thi (Methodology & Scope Transparency):**
>
> - **Phương pháp kiểm thử:** Khác với phân hệ Web (được chạy trực tiếp trên trình duyệt Chrome), đối với phân hệ Mobile App (Pool D), Tester áp dụng kỹ thuật **Phân tích mã nguồn hộp trắng (White-box Source Code Inspection)** trên toàn bộ file logic giao diện [`frontend-mobile/App.js`] (file:///c:/Coding/SoftwareTesting/Seminar/src/eshop-sut/frontend-mobile/App.js) kết hợp **Kiểm thử tích hợp các API Backend liên quan** (`/api/checkout`, `/api/orders`, `/api/products`).
> - **Lý do kỹ thuật & Phạm vi:** Dự án `frontend-mobile` chưa cài đặt `node_modules` và không khởi chạy máy ảo giả lập Expo. Mã nguồn `App.js` thể hiện rõ các đoạn mã lỗi được cài cắm có chủ đích (Deliberate Defect Injection) của SUT như: `parsed + 1` (dòng 620), `cart.slice(0, -1)` (dòng 391), hardcode IP LAN (dòng 16), và thiếu confirm dialog (dòng 155). Toàn bộ hành vi của giỏ hàng, cập nhật state React và payload gửi qua mạng được phân tích và chứng minh trực tiếp từ mã nguồn và dữ liệu thực tế tại Backend.

---

1. Cơ sở Đặc tả Nghiệp vụ (SRS Baseline)

Trích xuất trực tiếp từ tài liệu đặc tả [specs/system_requirements_specification.md] (file:///c:/Coding/SoftwareTesting/Seminar/src/eshop-sut/specs/system_requirements_specification.md) (Mục 4 - Giỏ hàng & Thanh toán):

FR-07: Giỏ hàng (Shopping Cart)

1. **Cấu trúc hiển thị:** Hiển thị danh sách sản phẩm với các thông tin: **Sản phẩm**, **Đơn giá**, **Số lượng** (có nút +/- để chỉnh), **Thành tiền** (`price * quantity`), và **Thao tác** (Xóa).
2. **Quy tắc gộp sản phẩm:** Thêm cùng một sản phẩm vào giỏ sẽ tăng số lượng, không được tạo dòng mới trùng lặp.
3. **Quy tắc Xóa sản phẩm:** Nút **Xóa sản phẩm** phải có **dialog xác nhận** (Confirmation Alert) trước khi thực hiện xóa khỏi giỏ.
4. **Điều hướng:** Có nút **Tiếp tục mua sắm** để quay về trang chủ.
5. **Quy chuẩn nhãn tổng tiền:** Tổng tiền hiển thị nhãn chính xác: **"Tổng cộng"** (nghiêm cấm hiển thị "Tổng tạm tính").
6. **Trạng thái giỏ rỗng:** Khi giỏ hàng không có sản phẩm, phải có hình minh họa và thông báo rõ ràng: "*Giỏ hàng của bạn đang trống*".

FR-08: Thanh toán (Checkout) liên kết Giỏ hàng

1. **Điều kiện đăng nhập:** Chỉ người dùng **đã đăng nhập** mới tiến hành thanh toán được; khách vãng lai phải được yêu cầu đăng nhập.

2. **Bảo toàn dữ liệu đặt hàng:** Toàn bộ danh sách sản phẩm trong giỏ hàng phải được gửi lên Backend đầy đủ, không được làm rơi rớt bất kỳ mặt hàng nào.
3. **Làm sạch sau thanh toán:** Sau khi thanh toán thành công, giỏ hàng phải được xóa sạch hoàn toàn (`setCart([])`).

---

2. Phần A: Domain Testing (Kiểm thử miền)

Bước 1: Xác định biến đầu vào & đầu ra (Inputs & Outputs)

Áp dụng Nguyên tắc Bước 1 (Slide S04):

STT	Tên biến	Kiểu dữ liệu	Bắt buộc	Ràng buộc nghiệp vụ (Constraints)
1	`product_id`	Integer	Có	ID sản phẩm hợp lệ tồn tại trong hệ thống.
2	`quantity` (Số lượng thêm)	Number	Có	Số nguyên dương ≥ 1; chỉ nhận số.
3	`edit_quantity` (Sửa số lượng)	Number	Có	Nhập trực tiếp trong giỏ; phải là số nguyên dương ≥ 1.
4	`cart_items` (Danh sách giỏ)	Array	-	Mảng chứa các sản phẩm: rỗng (0), đơn lẻ (1), hoặc nhiều (> 1).
5	`action` (Thao tác)	Enum	Có	Thêm từ Home, Thêm từ Detail, Sửa số lượng, Xóa món, Mua tiếp, Thanh toán.
6	`user_auth` (Xác thực)	Boolean	Có	Đã đăng nhập (`true` - có Token) hoặc Chưa đăng nhập (`false`).
Out 1	`line_subtotal`	Number (VND)	-	Thành tiền từng dòng: $\text{price} \times \text{quantity}$.
Out 2	`cart_total`	Number (VND)	-	Tổng tiền toàn bộ giỏ: $\sum(\text{price} \times \text{quantity})$.
Out 3	`badge_count`	Integer	-	Số lượng mặt hàng hiển thị trên Header: `Giỏ (N)`.
Out 4	Dialog xác nhận xóa	Alert Dialog	-	Hộp thoại hỏi người dùng xác nhận trước khi xóa.
Out 5	Điều hướng màn hình	Navigation	-	Chuyển sang `home`, `cart`, `checkout`, hoặc `login`.

Bước 2: Phân chia các lớp tương đương (Equivalence Partitioning)

2.1. Biến `quantity` khi thêm vào giỏ (Product Detail)

- **EC-ADDQTY-V1 (Valid):** Số nguyên dương ≥ 1. Đại diện: `1`, `2`.
- **EC-ADDQTY-I1 (Invalid):** Bằng 0 (`0`).
- **EC-ADDQTY-I2 (Invalid):** Số âm (`-1`).
- **EC-ADDQTY-I3 (Invalid - Format):** Chuỗi rỗng `""` hoặc ký tự chữ (`"abc"`).

2.2. Biến `edit_quantity` khi sửa trực tiếp trong giỏ hàng

- **EC-EDITQTY-V1 (Valid):** Sửa thành số nguyên dương ≥ 1. Đại diện: `2`, `5`.
- **EC-EDITQTY-I1 (Invalid):** Sửa thành 0 hoặc số âm.

- **EC-EDITQTY-I2 (Invalid):** Xóa trắng ô số lượng `""`.

2.3. Biến `cart_items` (Trạng thái danh sách giỏ hàng)

- **EC-CART-EMPTY (Valid):** Giỏ hàng rỗng (`cart.length = 0`).
- **EC-CART-SINGLE (Valid):** Giỏ hàng có đúng 1 sản phẩm (`cart.length = 1`).
- **EC-CART-MULTI (Valid):** Giỏ hàng có từ 2 sản phẩm trở lên (`cart.length > 1`).

2.4. Biến `delete_action` (Thao tác xóa sản phẩm)

- **EC-DEL-PROMPT (Valid):** Bấm Xóa → Kích hoạt Alert dialog xác nhận.
- **EC-DEL-CONFIRM (Valid):** Chọn Đồng ý → Xóa sản phẩm khỏi giỏ.
- **EC-DEL-CANCEL (Valid):** Chọn Hủy → Giữ nguyên sản phẩm trong giỏ.

2.5. Biến `user_auth` khi bấm Thanh toán

- **EC-CHK-AUTH (Valid):** Người dùng đã đăng nhập → Điều hướng sang màn hình Checkout (`view = 'checkout'`).
- **EC-CHK-GUEST (Invalid):** Khách chưa đăng nhập → Hiển thị Alert "Bạn cần đăng nhập để thanh toán!" và điều hướng sang Login (`view = 'login'`).

Bước 3: Thiết lập Bộ chuẩn Cô Lập Lỗi (Golden Positive Baseline per Slide S07)

> [!IMPORTANT] **Bộ Dữ Liệu Chuẩn Cô Lập Lỗi (Golden Positive Baseline Input):**
>
> - **Tài khoản đăng nhập:** `test@eshop.com` / `Test1234!` (Đã đăng nhập trên Mobile App, có Token).
> - **Sản phẩm kiểm thử chuẩn:** `iPhone 15 Pro Max` (ID: 1, Đơn giá: `30,000,000 ₫`).
> - **Số lượng thêm:** `1`.
> - **Thành tiền kỳ vọng:** `30,000,000 ₫`.
> - **Tổng cộng giỏ hàng kỳ vọng:** `30,000,000 ₫`.
> - **Trạng thái giỏ:** 1 sản phẩm đơn lẻ.
>
> **Kỹ thuật Cô lập lỗi (Single Fault Assumption):** Giữ nguyên sản phẩm và tài khoản chuẩn, chỉ biến đổi một yếu tố mục tiêu (ví dụ: thay đổi số lượng, thêm sản phẩm thứ hai, hoặc xóa sản phẩm) để xác định chính xác nguyên nhân lỗi.

Bước 4: Tinh gọn & Loại bỏ dư thừa (Domain Reduction Table)

STT	Cặp kiểm thử xem xét	Hành vi hệ thống	Quyết định tinh gọn	Lý do kỹ thuật
1	Thêm từ Home vs Thêm từ Detail	Cùng gọi hàm `addToCart`	**Giữ cả 2**	Thêm từ Home cố định $q = 1$; Detail cho phép nhập tùy chỉnh q.
2	Sửa số lượng $= 0$ vs Sửa số lượng âm	Chuẩn hóa số không hợp lệ	**Giữ cả 2**	`0` là điểm biên ranh giới; số âm kiểm tra việc xử lý dấu `-`.
3	Giỏ 1 món vs Giỏ 2 món khi checkout	Xử lý mảng `cart`	**Giữ cả 2**	Giỏ 2 món kích hoạt điều kiện rẽ nhánh lỗi `cart.length > 1`.

---

3. Phần B: Phân Tích Giá Trị Biên (Boundary Value Analysis - BVA)

Áp dụng phân tích giá trị biên cho 2 biến cốt lõi:

3.1. Phân tích biên biến `quantity` (Số lượng sản phẩm)

- **Biên âm sâu:** `-5` → Kỳ vọng: Chuẩn hóa về mức tối thiểu 1.
- $Boundary - 2$**:** `-1` → Kỳ vọng: Chuẩn hóa về 1.
- $Boundary - 1$**:** `0` → Kỳ vọng: Chuẩn hóa về 1.
- $BoundaryMin$ **(Biên tối thiểu hợp lệ):** `1` → Kỳ vọng: Chấp nhận số lượng đúng bằng 1.
- $Min + 1$**:** `2` → Kỳ vọng: Chấp nhận số lượng đúng bằng 2.
- $Nominal$**:** `5` → Kỳ vọng: Hợp lệ.
- $ExtremeUpper$**:** `9,999` → Kỳ vọng: Xử lý an toàn, tính đúng tổng tiền.

3.2. Phân tích biên biến `cart.length` (Số mặt hàng trong giỏ)

- Min **(Giỏ rỗng):** `0 món` → Hiển thị thông báo giỏ rỗng, ẩn nút thanh toán.
- $Min + 1$ **(Giỏ 1 món):** `1 món` → Hiển thị 1 dòng, tính đúng tổng tiền. Khi checkout gửi đủ 1 món.
- $BoundaryRẽnhánh$**:** `2 món` → **Điểm biên quyết định kiểm tra lỗi cắt xén sản phẩm (`cart.length > 1`)**.
- $Nominal$**:** `3 món` → Kiểm tra tính bảo toàn dữ liệu khi có nhiều sản phẩm.

---

4. Phần C: Thiết Kế & Thực Thi Bộ Test Case Chuẩn Hóa (Theo Mẫu Slide S07)

Bảng Thực Thi Test Case Chuẩn Hóa (Mẫu 6 cột Slide S07 Slide 8):

TC ID	Description (Test Objective) *(Syntax: Action + Function + Operating Condition)*	Pre-conditions & Test Steps	Expected Result	Observed / Actual Result	Status
TC-MOB-D01	**Verify Add Product to Cart with Valid Quantity from Detail Screen (SRS Baseline)**	**Pre-condition:** Đăng nhập `test@eshop.com`. Giỏ rỗng. **Test Data:** Product ID: 1 (`iPhone 15 Pro Max`), Quantity: `1`. **Steps:** 1. Mở chi tiết SP 1. 2. Nhập số lượng `1`. 3. Bấm 'Thêm vào giỏ hàng'. 4. Vào màn hình Giỏ hàng.	Thêm thành công; Giỏ có 1 SP; Số lượng: 1; Thành tiền: 30,000,000 ₫; Badge Header: `Giỏ (1)`.	**PASSED.** Sản phẩm hiển thị chính xác trong giỏ, tính đúng thành tiền 30,000,000 ₫ và cập nhật badge Header.	**PASSED**
TC-MOB-D02	**Verify Quick Add to Cart with Default Quantity 1 from Home Screen**	**Pre-condition:** Giỏ hàng rỗng. **Test Data:** Product ID: 2 (`Samsung Galaxy S24 Ultra`). **Steps:** 1. Tại Home screen, bấm nút 'Thêm vào giỏ' tại SP 2. 2. Vào Giỏ hàng.	Thêm thành công với số lượng mặc định là 1; Đơn giá: 28,000,000 ₫.	**PASSED.** Sản phẩm được thêm nhanh với `quantity = 1` chính xác.	**PASSED**

TC ID	Description (Test Objective) *(Syntax: Action + Function + Operating Condition)*	Pre-conditions & Test Steps	Expected Result	Observed / Actual Result	Status
TC-MOB-D03	**Verify Merge Quantity when Adding Duplicate Existing Product to Cart**	**Pre-condition:** Giỏ đã có SP 1 với số lượng 1. **Test Data:** Product ID: 1, Quantity thêm: 2. **Steps:** 1. Vào lại SP 1. 2. Nhập số lượng 2. 3. Bấm 'Thêm vào giỏ hàng'. 4. Kiểm tra giỏ hàng.	Giỏ vẫn chỉ có 1 dòng duy nhất cho SP 1; Số lượng tăng thành 3 ($1 + 2$); Thành tiền: 90,000,000 ₫.	**PASSED.** Hàm `addToCart` tìm thấy `existingIndex >= 0`, cộng dồn số lượng thành 3 và không tạo dòng mới.	**PASSED**
TC-MOB-D04	**Test Cart Quantity Direct Edit to Value 2 on Cart Screen**	**Pre-condition:** Giỏ có SP 1 với số lượng 1. **Test Data:** Gõ số 2 vào ô số lượng. **Steps:** 1. Mở Giỏ hàng. 2. Xóa số cũ, gõ số 2. 3. Quan sát số lượng và thành tiền.	Số lượng cập nhật thành đúng 2; Thành tiền: 60,000,000 ₫.	**FAILED.** SUT dính lỗi logic tại `App.js:620`: code tính `parsed + 1`, khiến khi gõ 2 thì số lượng bị **tự động tăng thành 3** ($2 + 1$); Thành tiền thành 90,000,000 ₫ (`BUG-MOB-01`).	**FAILED**
TC-MOB-D05	**Test Cart Quantity Direct Edit to Value 5 on Cart Screen**	**Pre-condition:** Giỏ có SP 1. **Test Data:** Gõ số 5 vào ô số lượng. **Steps:** 1. Mở Giỏ hàng. 2. Gõ số 5. 3. Quan sát số lượng.	Số lượng cập nhật thành đúng 5.	**FAILED.** Do lỗi `parsed + 1`, số lượng bị gán thành 6 ($5 + 1$) (`BUG-MOB-01`).	**FAILED**
TC-MOB-D06	**Verify Checkout with Multiple Items in Cart (Silent Item Dropping Bug)**	**Pre-condition:** Giỏ có **2 sản phẩm khác nhau**: SP 1 (iPhone 30tr) và SP 4 (AirPods 6tr). Tổng tiền: 36,000,000 ₫. **Test Data:** Cart length = 2. **Steps:** 1. Bấm 'Thanh toán'. 2. Bấm 'Xác Nhận Đơn Hàng'. 3.	Toàn bộ 2 sản phẩm được gửi lên Backend và lưu vào đơn hàng đầy đủ.	**FAILED.** File `App.js` dòng 391 cố tình viết `items: cart.length > 1 ? cart.slice(0, -1) : cart`, **cắt bỏ sản phẩm cuối cùng (AirPods Pro 2)** khỏi đơn hàng, trong khi khách hàng vẫn bị tính trọn vẹn 36,000,000 ₫ (`BUG-MOB-02`).	**FAILED**

TC ID	Description (Test Objective) *(Syntax: Action + Function + Operating Condition)*	Pre-conditions & Test Steps	Expected Result	Observed / Actual Result	Status
		Kiểm tra payload API gửi tới `/api/checkout`.			
TC-MOB-D07	**Validate Cart Item Deletion Confirmation Alert Prompt**	**Pre-condition:** Giỏ có 1 sản phẩm. **Test Data:** Bấm nút 'Xóa'. **Steps:** 1. Tại dòng sản phẩm trong giỏ, bấm 'Xóa'.	Ứng dụng hiển thị Alert dialog: "Bạn có chắc chắn muốn xóa sản phẩm này không?".	**FAILED.** Hàm `removeFromCart` gọi thẳng `splice(index, 1)` xóa ngay lập tức mà hoàn toàn không có Alert xác nhận theo yêu cầu FR-07 dòng 97 (`BUG-MOB-04`).	**FAILED**
TC-MOB-D08	**Verify Cart Total Label Standard Compliance (FR-07 Rule)**	**Pre-condition:** Giỏ có sản phẩm. **Test Data:** Quan sát nhãn tổng tiền. **Steps:** 1. Mở Giỏ hàng. 2. Đọc nhãn text phía trên nút Mua tiếp/Thanh toán.	Nhãn hiển thị chính xác là "Tổng cộng" theo đặc tả FR-07 dòng 99.	**FAILED.** Giao diện Mobile dòng 638 hiển thị nhãn: `"Tổng tạm tính: ..."` vi phạm quy chuẩn bắt buộc của đặc tả (`BUG-MOB-05`).	**FAILED**
TC-MOB-D09	**Validate Checkout Attempt as Unauthenticated Guest User**	**Pre-condition:** Khách chưa đăng nhập (`user = null`). Giỏ có 1 sản phẩm. **Test Data:** Bấm 'Thanh toán' tại Giỏ hàng. **Steps:** 1. Bấm nút Thanh toán.	Hiển thị Alert "Bạn cần đăng nhập để thanh toán!" và điều hướng sang màn hình Login.	**PASSED.** Hàm `openCheckout` kiểm tra `if (!user)` hiển thị Alert và chuyển `view = 'login'` chính xác.	**PASSED**
TC-MOB-D10	**Verify Empty Cart Screen UI and Continue Shopping Action**	**Pre-condition:** Giỏ hàng rỗng (`cart.length = 0`). **Test Data:** Mở màn hình Giỏ hàng. **Steps:** 1. Bấm vào icon Giỏ trên Navbar. 2. Bấm 'Tiếp tục mua sắm'.	Hiển thị thông báo "Giỏ hàng của bạn đang trống" và nút link; Bấm link quay về Home.	**PASSED.** Màn hình rỗng hiển thị đúng thông điệp và điều hướng về Home thành công.	**PASSED**

TC ID	Description (Test Objective) *(Syntax: Action + Function + Operating Condition)*	Pre-conditions & Test Steps	Expected Result	Observed / Actual Result	Status
TC-MOB-D11	**Validate Cart Clean-up after Successful Checkout**	**Pre-condition:** Đăng nhập hợp lệ, giỏ có 1 sản phẩm. **Test Data:** Thực hiện thanh toán hoàn tất. **Steps:** 1. Đặt hàng thành công. 2. Quay lại kiểm tra giỏ hàng.	Giỏ hàng được xóa sạch hoàn toàn (`cart = []`). Badge hiển thị `Giỏ (0)`.	**PASSED.** Hàm `handleConfirmCheckout` gọi `setCart([])`, giỏ hàng được reset sạch sau thanh toán.	**PASSED**
TC-MOB-D12	**Verify API URL Network Reachability Configuration**	**Pre-condition:** Kiểm tra mã nguồn cấu hình mạng. **Test Data:** Biến `API_URL`. **Steps:** 1. Đọc dòng 16 file `frontend-mobile/App.js`.	`API_URL` sử dụng biến môi trường hoặc cấu hình động linh hoạt.	**FAILED.** Mã nguồn hardcode cứng IP mạng nội bộ `http://192.168.10.13:3000/api`, gây mất kết nối hoàn toàn trên các môi trường mạng khác (`BUG-MOB-03`).	**FAILED**
TC-MOB-D13	**Verify Quantity Increment/Decrement (+/-) Buttons on Cart UI**	**Pre-condition:** Giỏ có 1 sản phẩm. **Test Data:** Quan sát các nút điều khiển số lượng. **Steps:** 1. Mở Giỏ hàng. 2. Tìm nút bấm [+] và [-].	Giao diện cung cấp nút [+] và [-] bên cạnh ô số lượng để thao tác chạm nhanh theo FR-07 dòng 95.	**FAILED.** Giao diện chỉ có 1 ô `TextInput` nhập liệu bàn phím, hoàn toàn thiếu 2 nút bấm [+] và [-] theo đặc tả (`BUG-MOB-06`).	**FAILED**
TC-MOB-D14	**Test Adding to Cart with Non-numeric Quantity String 'abc'**	**Pre-condition:** Mở chi tiết SP 1. **Test Data:** Quantity: `'abc'`. **Steps:** 1. Gõ 'abc' vào ô số lượng. 2. Bấm 'Thêm vào giỏ hàng'.	Hệ thống chuẩn hóa về số lượng hợp lệ tối thiểu là 1.	**PASSED.** Hàm `normalizeQuantity` dùng `parseInt('abc', 10)` trả về NaN, fallback an toàn về `1`.	**PASSED**
TC-MOB-D15	**Test Cart Subtotal Calculation Accuracy Across Multiple Items**	**Pre-condition:** Giỏ có SP 1 (30tr x 1) và SP 5 (4tr x 2). **Test Data:** Cart items: SP1 ($q = 1$	Dòng 1: 30,000,000 ₫; Dòng 2: 8,000,000 ₫; Tổng cộng: 38,000,000 ₫.	**PASSED.** Hàm `useMemo(cartTotal)` tính toán chính xác tổng tiền theo công thức số học.	**PASSED**

TC ID	Description (Test Objective) *(Syntax: Action + Function + Operating Condition)*	Pre-conditions & Test Steps	Expected Result	Observed / Actual Result	Status
		), SP5 ($q = 2$). **Steps:** 1. Mở Giỏ hàng. 2. So sánh tổng từng dòng và tổng toàn giỏ.			
TC-MOB-D16	**Verify Continue Shopping Button Navigation from Non-Empty Cart**	**Pre-condition:** Giỏ có 1 sản phẩm. **Test Data:** Bấm nút '← Mua tiếp'. **Steps:** 1. Tại chân trang Giỏ hàng, bấm nút '← Mua tiếp'.	Ứng dụng điều hướng quay lại trang chủ (Home), giỏ hàng giữ nguyên.	**PASSED.** Điều hướng về `view = 'home'`, trạng thái giỏ hàng được bảo lưu an toàn.	**PASSED**
TC-MOB-D17	**Test Order Cancellation from Mobile Order History**	**Pre-condition:** User có đơn hàng trạng thái `pending`. **Test Data:** Bấm 'Hủy đơn' tại đơn pending. **Steps:** 1. Vào Profile. 2. Bấm 'Hủy đơn'.	Hủy thành công, trạng thái đơn chuyển sang 'Đã hủy'.	**PASSED.** Gọi API `PUT /api/orders/:id/cancel` thành công, cập nhật trạng thái đơn hàng.	**PASSED**
TC-MOB-D18	**Verify Order Total Integrity Sent from Mobile Checkout**	**Pre-condition:** Giỏ có SP 30,000,000 ₫. **Test Data:** Payload gửi tới `/api/checkout`. **Steps:** 1. Thực hiện checkout. 2. Kiểm tra `total_amount` trong request body.	Backend phải tự tính lại tổng tiền từ giỏ hàng, không tin tưởng client theo FR-08 dòng 107.	**FAILED.** Mobile gửi trực tiếp `total_amount: finalAmount` và Backend nhận lưu thẳng vào CSDL mà không tự tính lại giá từ DB (`BUG-FR08-01`).	**FAILED**
TC-MOB-B01	**Verify Add Quantity at Negative Value (-1)**	**Pre-condition:** Mở chi tiết SP 1. **Test Data:** Quantity: `-1`. **Steps:** 1. Nhập `-1`. 2. Bấm 'Thêm vào giỏ hàng'.	Chuẩn hóa an toàn về mức tối thiểu 1.	**PASSED.** Hàm `normalizeQuantity` phát hiện `parsed <= 0`, chuẩn hóa thành `1`.	**PASSED**

TC ID	Description (Test Objective) *(Syntax: Action + Function + Operating Condition)*	Pre-conditions & Test Steps	Expected Result	Observed / Actual Result	Status
TC-MOB-B02	**Verify Add Quantity at Zero (0)**	**Pre-condition:** Mở chi tiết SP 1. **Test Data:** Quantity: `0`. **Steps:** 1. Nhập `0`. 2. Bấm 'Thêm vào giỏ hàng'.	Chuẩn hóa an toàn về mức tối thiểu 1.	**PASSED.** Hàm `normalizeQuantity` chuẩn hóa thành `1`.	**PASSED**
TC-MOB-B03	**Verify Add Quantity at Minimum Boundary (1 - Spec Min)**	**Pre-condition:** Mở chi tiết SP 1. **Test Data:** Quantity: `1`. **Steps:** 1. Nhập `1`. 2. Bấm 'Thêm vào giỏ hàng'.	Thêm thành công với số lượng đúng bằng 1.	**PASSED.** Hệ thống thêm chính xác với số lượng 1.	**PASSED**
TC-MOB-B04	**Verify Add Quantity at Minimum + 1 (2)**	**Pre-condition:** Mở chi tiết SP 1. **Test Data:** Quantity: `2`. **Steps:** 1. Nhập `2`. 2. Bấm 'Thêm vào giỏ hàng'.	Thêm thành công với số lượng đúng bằng 2.	**PASSED.** Hệ thống thêm chính xác với số lượng 2.	**PASSED**
TC-MOB-B05	**Verify Add Quantity at Extreme Upper Boundary (9,999)**	**Pre-condition:** Mở chi tiết SP 1. **Test Data:** Quantity: `9999`. **Steps:** 1. Nhập `9999`. 2. Bấm 'Thêm vào giỏ hàng'.	Xử lý an toàn; Thành tiền: 299,970,000,000 ₫.	**PASSED.** Hệ thống tính toán an toàn, không bị tràn số.	**PASSED**
TC-MOB-B06	**Verify Cart Edit Quantity with Empty String ''**	**Pre-condition:** Giỏ có 1 sản phẩm. **Test Data:** Xóa trắng ô số lượng (chuỗi `""`). **Steps:** 1. Mở Giỏ hàng. 2. Xóa hết số trong ô số lượng.	Tự động fallback về 1.	**PASSED.** `parseInt("", 10)` là NaN, code gán về giá trị an toàn là `1`.	**PASSED**
TC-MOB-B07	**Verify Cart Edit Quantity at Zero (0)**	**Pre-condition:** Giỏ có 1 sản phẩm. **Test Data:** Nhập số `0` vào ô số	Tự động fallback về 1 (không cho phép số lượng 0).	**PASSED.** Do điều kiện `parsed > 0` bị false, code gán về `1`.	**PASSED**

TC ID	Description (Test Objective) *(Syntax: Action + Function + Operating Condition)*	Pre-conditions & Test Steps	Expected Result	Observed / Actual Result	Status
		lượng. **Steps:** 1. Mở Giỏ hàng. 2. Gõ `0`.			
TC-MOB-B08	**Verify Cart Items Boundary at Min (0 items - Empty Cart)**	**Pre-condition:** Ứng dụng mới khởi động hoặc vừa xóa hết giỏ. **Test Data:** `cart.length = 0`. **Steps:** 1. Mở màn hình Giỏ hàng.	Hiển thị màn hình giỏ rỗng, ẩn bảng sản phẩm và nút đặt hàng.	**PASSED.** Giao diện render view `emptyBox` chính xác.	**PASSED**
TC-MOB-B09	**Verify Cart Items Boundary at 1 Item (Boundary Single Item)**	**Pre-condition:** Giỏ có đúng 1 sản phẩm. **Test Data:** `cart.length = 1`. **Steps:** 1. Mở Giỏ hàng. 2. Tiến hành Checkout.	Gửi đúng 1 sản phẩm lên Backend; Đơn hàng tạo đủ 1 sản phẩm.	**PASSED.** Do `cart.length` không lớn hơn 1, toán tử 3 ngôi gửi nguyên `cart` (không bị cắt xén).	**PASSED**
TC-MOB-B10	**Verify Cart Items Boundary at 2 Items (Critical Dropping Boundary)**	**Pre-condition:** Giỏ có đúng 2 sản phẩm khác nhau. **Test Data:** `cart.length = 2`. **Steps:** 1. Mở Giỏ hàng. 2. Tiến hành Checkout.	Cả 2 sản phẩm phải được đưa vào đơn hàng.	**FAILED.** Điều kiện `cart.length > 1` kích hoạt `cart.slice(0, -1)`, **cắt bỏ sản phẩm thứ 2** (`BUG-MOB-02`).	**FAILED**
TC-MOB-B11	**Verify Cart Items Boundary at 3 Items (Nominal Multi-item Cart)**	**Pre-condition:** Giỏ có đúng 3 sản phẩm khác nhau. **Test Data:** `cart.length = 3`. **Steps:** 1. Mở Giỏ hàng. 2. Tiến hành Checkout.	Toàn bộ 3 sản phẩm được tạo trong đơn hàng.	**FAILED.** `cart.slice(0, -1)` chỉ gửi 2 sản phẩm đầu, **cắt bỏ sản phẩm thứ 3** (`BUG-MOB-02`).	**FAILED**
TC-MOB-B12	**Verify Header Badge Count Boundary**	**Pre-condition:** Giỏ rỗng `Giỏ (0)`.	Text Header cập nhật từ `Giỏ`	**PASSED.** Text Header hiển thị đúng `Giỏ (${cart.length})`.	**PASSED**

TC ID	Description (Test Objective) *(Syntax: Action + Function + Operating Condition)*	Pre-conditions & Test Steps	Expected Result	Observed / Actual Result	Status
	when Cart Transitioning 0 -> 1	**Test Data:** Thêm 1 sản phẩm. **Steps:** 1. Thêm sản phẩm vào giỏ. 2. Quan sát text Header.	`(0)` thành `Giỏ (1)`.		

---

5. Phần D: Phân Tích Khoảng Trống Của AI (AI Gap Analysis)

Khi sử dụng các công cụ AI thông thường (ChatGPT/Claude/Copilot) chỉ với prompt chung chung *"viết test case kiểm thử giỏ hàng ứng dụng mobile EShop"*, AI thường bỏ sót các lỗi cực kỳ nghiêm trọng sau:

1. **Bỏ sót lỗi cắt xén sản phẩm khi thanh toán (`BUG-MOB-02`):**
   - *Thực tế:* File `App.js` dòng 391 cố tình viết `items: cart.length > 1 ? cart.slice(0, -1) : cart`. Khi giỏ hàng có từ 2 sản phẩm trở lên, sản phẩm cuối cùng bị **vứt bỏ âm thầm** khỏi payload đặt hàng trong khi tiền vẫn bị tính đủ. AI thông thường không bao giờ tưởng tượng lập trình viên có thể chèn một đoạn code phá hoại như vậy.
2. **Bỏ sót lỗi tự động tăng thêm 1 khi sửa số lượng trong giỏ (`BUG-MOB-01`):**
   - *Thực tế:* Dòng 620 file `App.js` viết `parsed + 1`. Người dùng muốn mua 2 cái thì ứng dụng tự ép thành 3 cái. AI nếu không đọc chi tiết hàm xử lý sự kiện `onChangeText` sẽ mặc định là form hoạt động bình thường.
3. **Bỏ sót lỗi hardcode IP mạng nội bộ (`BUG-MOB-03`):**
   - *Thực tế:* Dòng 16 `App.js` hardcode `http://192.168.10.13:3000/api`. Ứng dụng sẽ tê liệt hoàn toàn khi chạy ở bất kỳ máy hay mạng Wi-Fi nào khác.
4. **Bỏ sót việc thiếu hộp thoại xác nhận khi xóa sản phẩm (`BUG-MOB-04`):**
   - *Thực tế:* Thao tác bấm nút Xóa xóa ngay lập tức mà không gọi `Alert.alert` để xác nhận, vi phạm trực tiếp đặc tả FR-07 dòng 97 và nguyên lý trải nghiệm người dùng trên màn hình cảm ứng di động.
5. **Bỏ sót việc thiếu hoàn toàn cặp nút tăng/giảm số lượng (+/-) trên mobile UI (`BUG-MOB-06`):**
   - *Thực tế:* Đặc tả FR-07 dòng 95 quy định rõ phải có nút +/- để điều chỉnh số lượng. Giao diện mobile chỉ có 1 ô `TextInput` thô sơ và bắt buộc người dùng mở bàn phím ảo.

---

6. Phần E: Báo Cáo Lỗi Độc Lập (Chuẩn 10 Yếu Tố Bài Giảng Slide S08)

BUG-MOB-01: Giao diện Giỏ hàng tự động cộng thêm 1 khi người dùng chỉnh sửa số lượng

1. **Bug ID:** `BUG-MOB-01`
2. **Function name:** `FR-07: Cập nhật số lượng trong giỏ hàng (Mobile App UI)`
3. **Problem summary:** `Update Cart Quantity: Cart screen erroneously increments entered quantity by 1 (parsed + 1), making it impossible to set exact desired quantity.`
4. **How to reproduce the defect:**
   - **Environment:** Expo Mobile App (`frontend-mobile`), iOS/Android Simulator.
   - **Steps to Reproduce:**
     1. Thêm 1 sản phẩm vào giỏ hàng.
     2. Mở màn hình Giỏ hàng (`Giỏ`).
     3. Tại ô nhập số lượng, gõ số `2`.
     4. Quan sát giá trị hiển thị và thành tiền.
   - **Expected Behavior:** Số lượng cập nhật đúng bằng `2`, thành tiền bằng `đơn giá x 2`.
   - **Actual Behavior:** Số lượng bị ép thành `3` ($2 + 1$), thành tiền bị tính đội lên thành `đơn giá x 3`.

- **Root Cause & Code Reference:** Tại file `frontend-mobile/App.js` dòng 618-621:

```
newCart[index].quantity =
  Number.isFinite(parsed) && parsed > 0
    ? parsed + 1 // Cố tình cộng thêm 1
    : 1;
```

5. **Reported by:** Nguyễn Hoàng Anh Tú (MSSV: 23120183)
6. **Date:** 20/10/2026
7. **Assign to:** Mobile Developer
8. **Status:** `Open / New`
9. **Priority:** `Immediate (Critical)` — Làm sai lệch số lượng đặt mua của người dùng.
10. **Severity:** `Serious`

BUG-MOB-02: Ứng dụng Mobile âm thầm vứt bỏ sản phẩm cuối cùng khỏi đơn hàng khi giỏ hàng có nhiều sản phẩm

1. **Bug ID:** `BUG-MOB-02`
2. **Function name:** `FR-08: Đặt hàng từ Giỏ hàng trên Mobile (Mobile Checkout Flow)`
3. **Problem summary:** `Submit Order Items: Mobile app drops the last cart item (cart.slice(0, -1)) when cart has multiple items while charging full total.`
4. **How to reproduce the defect:**
   - **Environment:** Expo Mobile App & Backend API `:3000`.
   - **Steps to Reproduce:**
     1. Thêm 2 sản phẩm khác nhau vào giỏ (Ví dụ: iPhone 15 Pro Max 30tr và AirPods Pro 6tr). Tổng tiền: 36,000,000 ₫.
     2. Mở Giỏ hàng, bấm "Thanh toán".
     3. Bấm "Xác Nhận Đơn Hàng".
     4. Kiểm tra dữ liệu đơn hàng đã tạo trong lịch sử đơn hoặc CSDL bảng `orders`.
   - **Expected Behavior:** Đơn hàng lưu trọn vẹn cả 2 sản phẩm.
   - **Actual Behavior:** Khách hàng bị tính tiền trọn vẹn 36,000,000 ₫ nhưng payload gửi lên backend bị cắt mất sản phẩm AirPods Pro 2 do `cart.slice(0, -1)`. Khách trả tiền 2 món nhưng chỉ nhận 1 món!
   - **Root Cause & Code Reference:** Tại file `frontend-mobile/App.js` dòng 391:

```
body: JSON.stringify({
  items: cart.length > 1 ? cart.slice(0, -1) : cart, // Cắt bỏ phần tử cuối!
  total_amount: finalAmount,
  coupon_id: couponResult?.coupon_id || null,
}),
```

5. **Reported by:** Nguyễn Hoàng Anh Tú (MSSV: 23120183)
6. **Date:** 20/10/2026
7. **Assign to:** Mobile Developer
8. **Status:** `Open / New`
9. **Priority:** `Immediate (Critical)` — Lỗi gian lận nghiêm trọng làm thất thoát hàng hóa hoặc bớt xén quyền lợi người mua.
10. **Severity:** `Fatal`

BUG-MOB-03: Hardcode địa chỉ IP mạng nội bộ trong cấu hình API_URL

1. **Bug ID:** `BUG-MOB-03`
2. **Function name:** `Configuration: Cấu hình kết nối mạng (Mobile Network Config)`
3. **Problem summary:** `Configure API Endpoint: Mobile app hardcodes LAN private IP http://192.168.10.13:3000/api, breaking connectivity on other networks.`
4. **How to reproduce the defect:**
   - **Environment:** Expo Mobile App.
   - **Steps to Reproduce:**
     1. Chạy ứng dụng mobile trên máy hoặc mạng Wi-Fi khác với dải mạng `192.168.10.x`.
     2. Mở ứng dụng.
   - **Expected Behavior:** Ứng dụng đọc cấu hình từ file `.env` hoặc cho phép cấu hình URL động, kết nối tới backend thành công.
   - **Actual Behavior:** Mọi API gọi ra ngoài đều bị lỗi Network Request Failed, ứng dụng không tải được sản phẩm.
   - **Root Cause & Code Reference:** Tại file `frontend-mobile/App.js` dòng 16:

     ```
     const API_URL = "http://192.168.10.13:3000/api";
     ```

5. **Reported by:** Nguyễn Hoàng Anh Tú (MSSV: 23120183)
6. **Date:** 20/10/2026
7. **Assign to:** Mobile Lead Developer
8. **Status:** `Open / New`
9. **Priority:** `High`
10. **Severity:** `Fatal` — Khiến ứng dụng tê liệt hoàn toàn khi triển khai ngoài mạng nội bộ của dev.

BUG-MOB-04: Thao tác xóa sản phẩm trong giỏ thiếu hoàn toàn hộp thoại xác nhận (Confirmation Dialog)

1. **Bug ID:** `BUG-MOB-04`
2. **Function name:** `FR-07: Xóa sản phẩm khỏi giỏ hàng (Mobile App UI)`
3. **Problem summary:** `Confirm Cart Deletion: Deleting an item immediately removes it without Alert confirmation, violating FR-07 line 97.`
4. **How to reproduce the defect:**
   - **Environment:** Expo Mobile App.
   - **Steps to Reproduce:**
     1. Mở màn hình Giỏ hàng có sản phẩm.
     2. Nhấp chữ "Xóa" màu đỏ cạnh sản phẩm.
   - **Expected Behavior:** Xuất hiện Alert dialog hỏi "Bạn có chắc chắn muốn xóa sản phẩm này không?" với 2 lựa chọn [Hủy] và [Đồng ý] theo đặc tả FR-07 dòng 97.
   - **Actual Behavior:** Sản phẩm bị xóa ngay lập tức khỏi mảng `cart` mà không có bất kỳ bước xác nhận nào, gây nguy cơ bấm nhầm rất cao trên màn hình cảm ứng di động.
   - **Root Cause & Code Reference:** Tại file `frontend-mobile/App.js` dòng 155-159:

     ```
     const removeFromCart = (index) => {
       const newCart = [...cart];
       newCart.splice(index, 1);
       setCart(newCart);
     };
     ```

5. **Reported by:** Nguyễn Hoàng Anh Tú (MSSV: 23120183)
6. **Date:** 20/10/2026
7. **Assign to:** Mobile Developer
8. **Status:** `Open / New`

9. **Priority:** `Medium`
10. **Severity:** `Medium`

BUG-MOB-05: Nhãn tổng tiền hiển thị sai quy chuẩn đặc tả ("Tổng tạm tính" thay vì "Tổng cộng")

1. **Bug ID:** `BUG-MOB-05`
2. **Function name:** `FR-07: Hiển thị tổng tiền giỏ hàng (Mobile App UI / Label)`
3. **Problem summary:** `Enforce Label Standard: Cart screen displays 'Tổng tạm tính' instead of required 'Tổng cộng' label.`
4. **How to reproduce the defect:**
   - **Environment:** Expo Mobile App.
   - **Steps to Reproduce:**
     1. Mở Giỏ hàng có sản phẩm.
     2. Quan sát dòng hiển thị tổng tiền phía trên các nút bấm thao tác.
   - **Expected Behavior:** Nhãn hiển thị là `"Tổng cộng: ..."` theo đúng yêu cầu tại đặc tả FR-07 dòng 99.
   - **Actual Behavior:** Nhãn hiển thị là `"Tổng tạm tính: ..."`.
   - **Root Cause & Code Reference:** Tại file `frontend-mobile/App.js` dòng 637-639:

```
<Text style={styles.totalText}>
  Tổng tạm tính: {formatMoney(cartTotal)}
</Text>
```

5. **Reported by:** Nguyễn Hoàng Anh Tú (MSSV: 23120183)
6. **Date:** 20/10/2026
7. **Assign to:** Mobile Developer
8. **Status:** `Open / New`
9. **Priority:** `Low`
10. **Severity:** `Cosmetic`

BUG-MOB-06: Giao diện Giỏ hàng mobile thiếu hoàn toàn các nút tăng/giảm (+/-) số lượng

1. **Bug ID:** `BUG-MOB-06`
2. **Function name:** `FR-07: Bộ điều khiển số lượng trên Mobile (Mobile UI / UX)`
3. **Problem summary:** `Provide Quantity Controls: Cart screen lacks +/- buttons, forcing manual virtual keyboard input.`
4. **How to reproduce the defect:**
   - **Environment:** Expo Mobile App.
   - **Steps to Reproduce:**
     1. Mở Giỏ hàng có sản phẩm.
     2. Quan sát khu vực điều chỉnh số lượng của từng mặt hàng.
   - **Expected Behavior:** Có 2 nút bấm [+] và [-] đặt cạnh ô số lượng để người dùng chạm nhanh theo đặc tả FR-07 dòng 95.
   - **Actual Behavior:** Chỉ hiển thị 1 ô `TextInput` nhập chữ số, buộc người dùng phải kích hoạt bàn phím ảo di động và dính lỗi `BUG-MOB-01`.
   - **Root Cause & Code Reference:** Tại file `frontend-mobile/App.js` dòng 609-625 chỉ render thẻ `<TextInput style={styles.cartQuantityInput} ... />`.
5. **Reported by:** Nguyễn Hoàng Anh Tú (MSSV: 23120183)
6. **Date:** 20/10/2026
7. **Assign to:** Mobile Developer
8. **Status:** `Open / New`
9. **Priority:** `Medium`

10. **Severity:** `Medium`

---

7. Phần F: Báo Cáo Tổng Kết Kiểm Thử (Test Summary Report - Chuẩn Slide S08)

7.1. Bảng Thống Kê Theo Chức Năng (Statistics by Function - Slide S08 Slide 18)

No	Items (Function)	Tested	Passed	Failed	Blocked	Skipped	Total TCs	Test Coverage
1	**Mobile Cart & Checkout (Pool D)**	**30**	**20**	**10**	**0**	**0**	**30**	**100%**
	— Domain Testing Cases	18	11	7	0	0	18	100%
	— Boundary Value Analysis (BVA)	12	9	3	0	0	12	100%

7.2. Bảng Thống Kê Lỗi Theo Bản Chất Kỹ Thuật (Statistics by Defect Type - Slide S08 Slide 19)

Defect Type (Bản chất lỗi)	Fatal	Serious	Medium	Cosmetic	Tổng số Lỗi	Tỷ lệ (%)
Data Integrity & Fraud (Cắt xén sản phẩm `slice(0,-1)` khi thanh toán)	1	0	0	0	**1**	16.7%
Network & Configuration (Hardcode IP LAN)	1	0	0	0	**1**	16.7%
Business Logic (Tự động cộng thêm 1 khi sửa số lượng)	0	1	0	0	**1**	16.7%
Missing Feature (Thiếu hộp thoại xác nhận xóa, thiếu nút +/-)	0	0	2	0	**2**	33.3%
User Interface (Nhãn sai chuẩn 'Tổng tạm tính')	0	0	0	1	**1**	16.6%
TỔNG CỘNG	**2**	**1**	**2**	**1**	**6 Bugs**	**100%**

7.3. Bảng Thống Kê Lỗi Theo Mức Độ Nghiêm Trọng (Statistics by Defect Severity - Slide S08 Slide 21)

Mức độ nghiêm trọng (Severity)	Định nghĩa theo bài giảng S08	Số lượng phát hiện	Tỷ lệ (%)
Fatal	Cắt xén rơi rớt sản phẩm của khách hàng (`slice(0,-1)`), hardcode IP gây sập kết nối	2	33.3%
Serious	Lỗi tự động cộng thêm 1 khi sửa số lượng khiến người dùng không thể mua đúng số lượng mong muốn	1	16.7%
Medium	Thiếu dialog xác nhận xóa sản phẩm, thiếu nút bấm tăng/giảm +/- trên mobile UI	2	33.3%
Cosmetic	Sai lệch nhãn hiển thị giao diện ("Tổng tạm tính" thay vì "Tổng cộng")	1	16.7%
TỔNG CỘNG		**6 Bugs**	**100%**